Unit 05: Operator Overloading

CONTENT

Objectives

After studying this unit, you will be able to:

- Recognize the operator overloading
- Describe the rules for operator overloading
- Explain the overloading of unary operators
- Discuss the various binary operators with friend function and member function

Introduction

C++ comes with a large number of operators. Many of these operators have previously been defined and used in earlier units. Operator overloading is one of C++'s unique features. In a programming language that supports object-oriented features, this characteristic is required.

Overloading an operator simply means attaching additional meaning and semantics to an operator. It enables an operator to exhibit more than one operation polymorphically, as illustrated below:

You're probably aware that the addition operator (+) is primarily a numeric operator, requiring two number operands. It returns a numeric number equal to the product of the two operands. Obviously, this can't be used to join two strings together. We may, however, extend the addition operator's operation to include string concatenation. As a result, the addition operator would work like this:

"PROG"+"RAM"

Should produce a single string

"PROGRAM"

This act of redefining the effect of an operator is called operator overloading. The original meaning and action of the operator however remains as it is. Only an additional meaning is added to it.

Function overloading allows different functions with different argument list having the same name. Similarly an operator can be redefined to perform additional tasks.

Operator overloading is accomplished using a special function, which can be a member function or friend function. The general syntax of operator overloading is:

<return_type> operator <operator_being_overloaded>(<argument list>);

To overload the addition operator (+) to concatenate two characters, the following declaration, which could be either member or friend function, would be needed:

char * operator + (char *s2);

Notes

1. The purpose of operator overloading is to provide a special meaning of an operator for a user-defined data type.
2. Using operator overloading, we can redefine the majority of C++ operator.
3. Using operator overloading, we can specify more than one meaning for an operator in one scope.

5.1 Defining Operator Overloading

The overloading principle applies to both functions and operators in C++. That is, operators can be expanded to deal with classes as well as built-in types. By overloading the built-in operator to execute a specific computation when the operator is used on objects of that class, a programmer can provide his or her own operator to a class. Is it true that operator overloading is useful in real-world applications? It most certainly can, making writing code that seems natural a breeze.

Operator overloading, on the other hand, like any advanced C++ feature, adds to the language's complexity. Furthermore, because operators have a much-defined meaning and most programmers don't expect them to do a lot of work, overloading operators can be exploited to make code incomprehensible. But we're not going to do it.

Did you know?

What is the advantage of operator overloading?

By using operator overloading we can easily access the objects to perform any operations.

An Example of Operator Overloading

Complex a (1.2, 1.3); //this class is used to represent complex numbers

Complex b (2.1, 3); //notice the construction taking 2 parameters for the real and imaginary part

Complex c = a+b; //for this to work the addition operator must be overloaded

The addition without having overloaded operator + could look like this:

Complex c = a.Add(b);

This piece of code is not as readable as the first example though—we're dealing with numbers, so doing addition should be natural. (In contrast to cases when programmers abuse this technique, when the concept represented by the class is not related to the operator—like using + and - to add and remove elements from a data structure. In this cases operator overloading is a bad idea, creating confusion.)

In order to allow operations like Complex c = a+b, in above code we overload the "+" operator. The overloading syntax is quite simple, similar to function overloading, the keyword operator must be followed by the operator we want to overload:

```
class Complex
{
public:
Complex(double re,doubleim)
:real(re),imag(im)
{};
Complex operator+(const Complex& other);
Complex operator=(const Complex& other);
private:
double real;
doubleimag;
};
Complex Complex::operator+(const Complex& other)
{
doubleresult_real = real + other.real;
doubleresult_imaginary = imag + other.imag;
return Complex( result_real, result_imaginary );
}
```

The assignment operator can be overloaded similarly. Notice that we did not have to call any accessor functions in order to get the real and imaginary parts from the parameter other since the overloaded operator is a member of the class and has full access to all private data.

Alternatively, we could have defined the addition operator globally and called a member to do the actual work. In that case, we'd also have to make the method a friend of the class, or use anaccessor method to get at the private data:

```
friend Complex operator+(Complex);
Complex operator+(const Complex &num1, const Complex &num2)
{
doubleresult_real = num1.real + num2.real;
doubleresult_imaginary = num1.imag + num2.imag;
return Complex( result_real, result_imaginary );
}
```

Why would you do this? when the operator is a class member, the first object in the expression must be of that particular type. It's as if you were writing:

```
Complex a( 1, 2 );
Complex a( 2, 2 );
Complex c = a.operator=( b );
```

When it's a global function, the implicit or user-defined conversion can allow the operator to act even if the first operand is not exactly of the same type:

```
Complex c = 2+b; //if the integer 2 can be converted by the Complex
class, this expression is valid
```

By the way, the number of operands to a function is fixed; that is, a binary operator takes two operands, a unary only one, and you can't change it. The same is true for the precedence of operators too; for example the multiplication operator is called before addition. There are some

operators that need the first operand to be assignable, such as : operator=, operator(), operator[] and operator->, so their use is restricted just as member functions (non-static), they can't be overloaded globally. The operator=, operator& and operator, (sequencing) have already defined meanings by default for all objects, but their meanings can be changed by overloading or erased by making them private.

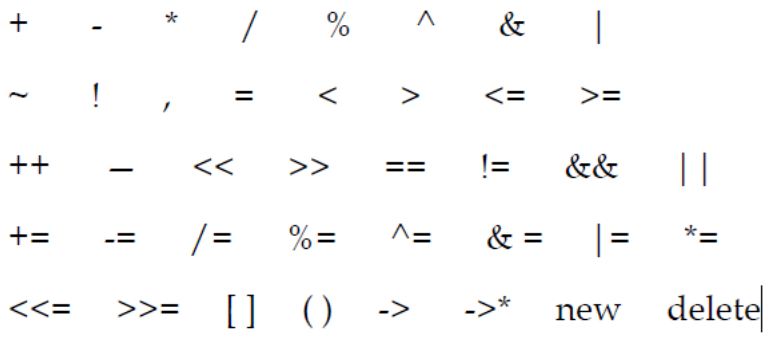
Another intuitive meaning of the "+" operator from the STL string class which is overloaded to do concatenation:

string prefix("de");

string word("composed");

string composed = prefix+word;

Using "+" to concatenate is also allowed in Java, but note that this is not extensible to other classes, and it's not a user defined behavior. Almost all operators can be overloaded in C++:



The only operators that can't be overloaded are the operators for scope resolution (::), member selection (.), and member selection through a pointer to a function(.*). Overloading assumes you specify a behavior for an operator that acts on a user defined type and it can't be used just with general pointers. The standard behavior of operators for built-in (primitive) types cannot be changed by overloading, that is, you can't overload operator+(int,int).

will not evaluate all three operations and will stop after a false one is found. This behavior does not apply to operators that are overloaded by the programmer.

Even the simplest C++ application, like a "hello world" program, is using overloaded operators. This is due to the use of this technique almost everywhere in the standard library (STL). Actually the most basic operations in C++ are done with overloaded operators, the IO(input/output) operators are overloaded versions of shift operators(<<, >>). Their use comes naturally to many beginning programmers, but their implementation is not straightforward. However a general format for overloading the input/output operators must be known by any C++ developer. We will apply this general form to manage the input/output for our Complex class:

```
friendostream&operator<<(ostream&out, Complex c) //output
{
out<<"real part: "<<real<<"\n";
out<<"imag part: "<<imag<<"\n";
return out;
}
friendistream&operator>>(istream&in, Complex &c) //input
{
cout<<"enter real part:\n";
in>>c.real;
cout<<"enter imag part: \n";
in>>c.imag;
```

return in;

}

A important trick that can be seen in this general way of overloading IO is the returning reference for istream/ostream which is needed in order to use them in a recursive manner:

Complex a(2,3);

Complex b(5.3,6);

cout<<a<<b;

5.2 Rules for Overloading Operator

To overload any operator, we must first comprehend the rules that apply. Let's go over some of the ones that have already been discussed.

Notes

1. For it to work, at least one operand must be a user-defined class object.
2. We can only overload existing operators. We can't overload new operators.
3. Some operators cannot be overloaded using a friend function. However, such operators can be overloaded using member function.

Following are the operators that cannot be overloaded.

Operator	Purpose
.	Class member access operator
.*	Class member access operator
::	Scope Resolution Operator
?:	Conditional Operator
Sizeof	Size in bytes operator
#	Preprocessor Directive
=	Assignment operator
0	Function call operator
0	Subscripting operator
->	Class member access operator

1. Operators already predefined in the C++ compiler can be only overloaded. Operator cannot change operator templates that is for example the increment operator ++ is used only as unary operator. It cannot be used as binary operator.

2. Overloading an operator does not change its basic meaning. For example assume the + operator can be overloaded to subtract two objects. But the code becomes unreachable.

class integer

{intx, y;

public:

int operator + ();

}

int integer: : operator + ( )

{

return (x-y);

}

3. Unary operators, overloaded by means of a member function, take no explicit argument and return no explicit values. But, those overloaded by means of a friend function take one reference argument (the object of the relevant class).

4. Binary operators overloaded through a member function take one explicit argument and those which are overloaded through a friend function take two explicit arguments.

Operator to Overload	Arguments passed to the Member Function	Argument passed to the Friend Function
Unary Operator	No	1
Binary Operator	1	2

5. Overloaded operators must either be a non-static class member function or a global function. A global function that needs access to private or protected class members must be declared as a friend of that class. A global function must take at least one argument that is of class or enumerated type or that is a reference to a class or enumerated type.

5.3 Overloading Unary Operators

When a unary operator is overloaded with a member function, no arguments are provided to the function, but a friend function requires only one input.

Did you know? **Unary operators operate on a single operand.**

Example

- **The increment (++) and decrement (--) operators.**
- **The unary minus (-) operator.**
- **The logical not (!) operator.**

// Program

```
#include<iostream>
using namespace std;
class sample{
private:
int x;
public:
sample(){
x=0;
}
void operator ++ (){
++x;
```

```
}
voiddisp(){
cout<<"Value of X is"<< x <<endl;
}
};
main(){
sampleobj;
obj.disp();
++obj;
obj.disp();
}
```

Output

```
Value of X is0
Value of X is1

Process returned 0 (0x0)   execution time : 0.052 s
Press any key to continue.
```

5.4 Overloading Binary Operator

Those operators which operate on two operands or data are called binary operators.Binary operators can be overloaded using C++.This will be better understood by means of the following program.

//Program

```
#include<iostream>
using namespace std;
class Complex{
inta,b;
public:
voidget_data(){

cout<<"Enter the value of complex numbers a and b";
cin>>a>>b;
  }
Complex operator +(Complex ob){
Complex c;
c.a=a+ob.a;
c.b=b+ob.b;
return (c);
}
void display()
```

```
{
cout<< a << "+" << b << "i" << "\n";
}
};
main(){
Complex obj1,obj2,result;
obj1.get_data();
obj1.display();
obj2.get_data();
result=obj1+obj2;
result.display();
}
```

Output

```
Enter the value of complex numbers a and b 12 2
12+2i
Enter the value of complex numbers a and b 12 2
24+4i

Process returned 0 (0x0)   execution time : 6.215 s
Press any key to continue.
```

5.5 Using Friend Function

- Friend function using operator overloading offers better flexibility to the class.
- These functions are not a members of the class and they do not have 'this' pointer.
- Overload a unary operator you have to pass one argument.
- Overload a binary operator you have to pass two arguments.
- Friend function can access private members of a class directly.

Now we are going to discuss what is a friend function in C++a friend function of a class is defined outside, and all of the private members we can access usingthe friend function outside the class prototype for friend functions appear in the class definition.If we are going for the return type of the function, we are going to use a friend keyword thereand after that we are writing the return type of the function, and friend function can access all theprivate members of the class also outside the class using the keyword friend compiler knows thegiven function is friend function by when we are using to the declared keyword there, itrepresents to the compiler that we are using the friend function in our code. Now we have someof the using friend functions in C++ why we are using the friend function and why friendfunctions are used to access the members in C++.

Notes

A friend function of a class is defined outside that class' scope but it has the right to access all private and protected members of the class.

```
//Program
#include <iostream>
```

```
using namespace std;
class sample
{
private:
int number;
public:
sample(): number(0) { }
friendintprintNumber(sample);
};
intprintNumber(sample s)
{
s.number += 100;
returns.number;
}
int main()
{
sample s;
cout<<"Entered Number is : "<<printNumber(s)<<endl;
return 0;
}
```

Output

```
Entered Number is : 100

Process returned 0 (0x0)   execution time : 0.013 s
Press any key to continue.
```

Task: Analyze the uses of friend function in operator overloading.

5.6 Using Member Function

You learned in the unit on overloading the arithmetic operators that it's ideal to implement the overloaded operator as a friend function of the class when it doesn't affect its operands. We usually overload the operator with a class member function for operators that modify their operands.

Overloading operators using a member function is very similar to overloading operators usinga friend function. When overloading an operator using a member function:

1. The leftmost operand of the overloaded operator must be an object of the class type.

2. The leftmost operand becomes the implicit *this parameter. All other operands becomefunction parameters.

Most operators can actually be overloaded either way, however there are a few exception cases:

3. If the leftmost operand is not a member of the class type, such as when overloadingoperator+(int, YourClass), or operator<<(ostream&, YourClass), the operator must beoverloaded as a friend.

4. The assignment (=), subscript ([]), call (()), and member selection (->) operators must beoverloaded as member functions.

Overloading the unary negative (–) operator

The negative operator is a unary operator that can be implemented using either method. Beforee show you how to overload the operator using a member function, here's a reminder of howwe overloaded it using a friend function:

```
class Cents
{
private:
intm_nCents;
public:
Cents(intnCents) { m_nCents = nCents; }
// Overload -cCents
friend Cents operator-(const Cents &cCents);
};
// note: this function is not a member function!
Cents operator-(const Cents &cCents)
{
return Cents(-cCents.m_nCents);
}
```

Now let's overload the same operator using a member function instead:

```
class Cents
{
private:
intm_nCents;
public:
Cents(intnCents) { m_nCents = nCents; }
// Overload -cCents
Cents operator-();
};
// note: this function is a member function!
Cents Cents::operator-()
{
return Cents(-m_nCents);
}
```

 Caution: Remember that when C++ sees the function prototype Cents Cents::operator-();,the compiler internally converts this to Cents operator-(const Cents *this), which you willnote is almost identical to our friend version Cents operator-(const Cents &cCents)!

You'll notice that this procedure is very similar to the previous one. The member function version of operator, on the other hand, does not take any parameters! What happened to the parameter? You learnt that a member function has an implicit *this pointer that always points to the class object the member function is working on in the course on the hidden this pointer. In the member function version, the parameter we had to explicitly list in the friend function version (which doesn't have a *this pointer) becomes the implicit *this parameter.

Overloading the binary addition (+) operator

Let's take a look at an example of a binary operator overloaded both ways. First, overloadingoperator+ using the friend function:

```
class Cents
{
private:
intm_nCents;
public:
Cents(intnCents) { m_nCents = nCents; }
// Overload cCents + int
friend Cents operator+(Cents &cCents, intnCents);
intGetCents() { return m_nCents; }
};
// note: this function is not a member function!
Cents operator+(Cents &cCents, intnCents)
{
return Cents(cCents.m_nCents + nCents);
}
```

Now, the same operator overloaded using the member function method:

```
class Cents
{
private:
intm_nCents;
public:
Cents(intnCents) { m_nCents = nCents; }
// Overload cCents + int
Cents operator+(intnCents);
intGetCents() { return m_nCents; }
};
// note: this function is a member function!
Cents Cents::operator+(intnCents)
{
return Cents(m_nCents + nCents);
}
```

Our two-parameter friend function becomes a one-parameter member function, because theleftmost parameter (cCents) becomes the implicit *this parameter in the member function version.

Most programmers find the friend function version easier to read than the member functionversion, because the parameters are listed explicitly. Furthermore, the friend function versioncan be used to overload some things the member function version cannot. For example, friendoperator+(int, cCents) cannot be converted into a member function because the leftmost parameteris not a class object.

However, when dealing with operands that modify the class itself (eg. operators =, +=, -=, ++,-, etc…) the member function method is typically used because C++ programmers are used towriting member functions (such as access functions) to modify private member variables. Writingfriend functions that modify private member variables of a class is generally not consideredgood coding style, as it violates encapsulation.

5.7 Manipulation of Strings using Operator Overloading

ANSI C implements strings using character assays, pointers and string functions. There are nooperators for manipulating the strings. There is no direct operator that could act upon thestrings. Although, these limitations exist in C++ as well, it permits us to create our own definitionsof operators that can be used to manipulate the strings very much similar to the decimal numbersfor e.g. we shall be able to use the statement like

String3 = string1 + string2;

The following program to overload the '+' operator to append one string to another

```
//Program
#include<iostream.h>
#include<conio.h>
classstr
{
char *name;
public:
str();
str(char *);
void get();
void show();
str operator + (str);
};

str::str() Notes
{
name = new char[1];
}
str::str(char *a)
{
inti = strlen(a);
name = new char[i + 1];
strcpy(name, a);
}
```

```
strstr::operator + (str s)
{
inti = strlen(name) + strlen(s.name);
strtmp;
tmp.name = new char[i+1];
strcpy(tmp.name, name);
strcat(tmp.name, s.name);
returntmp;
}
voidstr::get()
{
cin>> name;
}
voidstr::show()
{
cout<< name;
}
void main()
{
clrscr();
str S3;
str S1("hello");
str S2("monty");
S3 = S1 + S2;
S3.show();
}
```

Summary

- In this unit, we have seen how the normal C++ operators can be given new meaningswhen applied to user-defined data types.
- The keyword operator is used to overload an operator, and the resulting operator willadopt the meaning supplied by the programmer.
- Closely related to operator overloading is the issue of type conversion. Some conversionstake place between user defined types and basic types.
- We have seen that how friend function is working with different situations.
- Two approaches are used in such conversion: A one argument constructor changes a basictype to a user defined type, and a conversion operator converts a user-defined type to abasic type.
- When one user-defined type is converted to another, either approach can be used.

Keywords

Operator Overloading: Attaching additional meaning and semantics to an operator. It enablesto exhibit more than one operations polymorphically.

Strings: The C++ strings library provides the definitions of the basic_string class, which is a classtemplate specifically designed to manipulate strings of characters of any character type.

Unary Operators: Unary operators operate on one operand (variable or constant). There are twotypes of unary operators- increment and decrement.

Self Assessment

1. Which is the correct statement about operator overloading in C++?

A. Only arithmetic operators can be overloaded

B. Associativity and precedence of operators does not change

C. Precedence of operators are changed after overloading

D. Only non-arithmetic operators can be overloaded

2. Which of the following operators cannot be overloaded?

A. .* (Pointer-to-member Operator)

B. :: (Scope Resolution Operator)

C. .* (Pointer-to-member Operator)

D. All of the above

3. Operator overloading is also called …….. polymorphism.

A. Run time

B. Initial time

C. Compile time

D. Completion time

4. Which one is not unary operator

A. @

B. ++

C. –

D. - , !

5. …… overloaded by means of a member function, take no explicit arguments and return no explicit values.

A. Unary operators

B. Binary operators

C. Arithmetic operators

D. Function operator

6. Those operators which operate on two operands or data are called binary operators.

A. True

B. False

7. we can define a binary operator as :

A. The operator that performs its action on two operand
B. The operator that performs its action on three operand
C. The operator that performs its action on any number of operands
D. The operator that performs its action on a single operand

8. Correct example of a binary operator is _____.

A. –
B. +
C. ++
D. Dereferencing operator (*)

9. Overloading the addition (+) operator is correct function name of :

A. operator (+).
B. operator_+.
C. operator+.
D. operator:+.

10. What is the syntax of overloading operator + for class A?

A. A operator+(argument_list){}
B. A operator[+](argument_list){}
C. int +(argument_list){}
D. int [+](argument_list){}

11. What is a friend function in C++?

A. A function which can access all the private, protected and public members of a class
B. A function which is not allowed to access any member of any class
C. A function which is allowed to access public and protected members of a class
D. A function which is allowed to access only public members of a class

12. Pick the correct statement.

A. Friend functions are in the scope of a class
B. Friend functions can be called using class objects
C. Friend functions can be invoked as a normal function
D. Friend functions can access only protected members not the private members

13. Which keyword is used to represent a friend function?

A. friend
B. Friend
C. friend_func
D. Friend_func

14. What is the syntax of friend function?

A. friend class1 Class2;
B. friend class;
C. friend class
D. friend class()

15. Which operator is overload in following program

```
#include <iostream>
using namespace std;
class sample {
  int x, y;
public:
  sample() {}
  sample(intsx, intsy) {
    x = sx;
    y = sy;
  } void
show() {
    cout<< x << " ";
    cout<< y << "\n";
  }
friend sample operator+(sample ob1, sample ob2);
};
sample operator+(sample ob1, sample ob2)
{
  sample temp;

  temp.x = ob1.x + ob2.x;
  temp.y = ob1.y + ob2.y;

  return temp;
}
int main()
{
  sample ob1(10, 20), ob2( 5, 30);
  ob1 = ob1 + ob2;
  ob1.show();
  return 0;
}
```

A. Overloading function
B. Binary operator overloading using friend function
C. Unary operator overloading
D. None of above

Answers for Self Assessment

1.	B	2.	D	3.	C	4.	A	5.	A
6.	A	7.	B	8.	B	9.	A	10.	A

11. A 12. C 13. A 14. A 15. B

Review Questions

1. Overload the addition operator (+) to assign binary addition. The following operationshould be supported by +.

110010 + 011101 = 1001111

2. Write a program that demonstrate working of binary operator overloading.

3. Which operators are not allowed to be overloaded?

4. What are the differences between overloading a unary operator and that of a binaryoperator? Illustrate with suitable examples.

5. Why is it necessary to convert one data type to another? Illustrate with suitable examples.

6. How many arguments are required in the definition of an overloaded unary operator?

7. When used in prefix form, what does the overloaded + + operator do differently fromwhat it does in postfix form?

8. Explain in detail manipulation of strings using operator overloading.

9. Write a note on unary operators.

10. What are the various rules for overloading operators?

Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata McGraw-Hill.

Herbert Schildt; The Complete Reference C++; Tata McGraw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

Web Links

http://msdn.microsoft.com/en-us/library/wcz57btd(v=vs.80).aspx

http://www.learncpp.com/cpp-tutorial/117-multiple-inheritance/

https://www.codecademy.com/learn/learn-c-plus-plus